

Drakens Energy 360

An end-to-end synthetic data engineering platform for a downstream energy business

Report generated 2026-09-08 · Warehouse: drakens360_test_full.duckdb

Independent synthetic portfolio project. Drakens Energy is a fictional company invented for this project. It does not exist. Every site, customer, employee, supplier, asset, coordinate, price, volume, margin and incident in this report is randomly generated. Site coordinates are South African town centroids plus random jitter and do not represent any real service station.

Summary

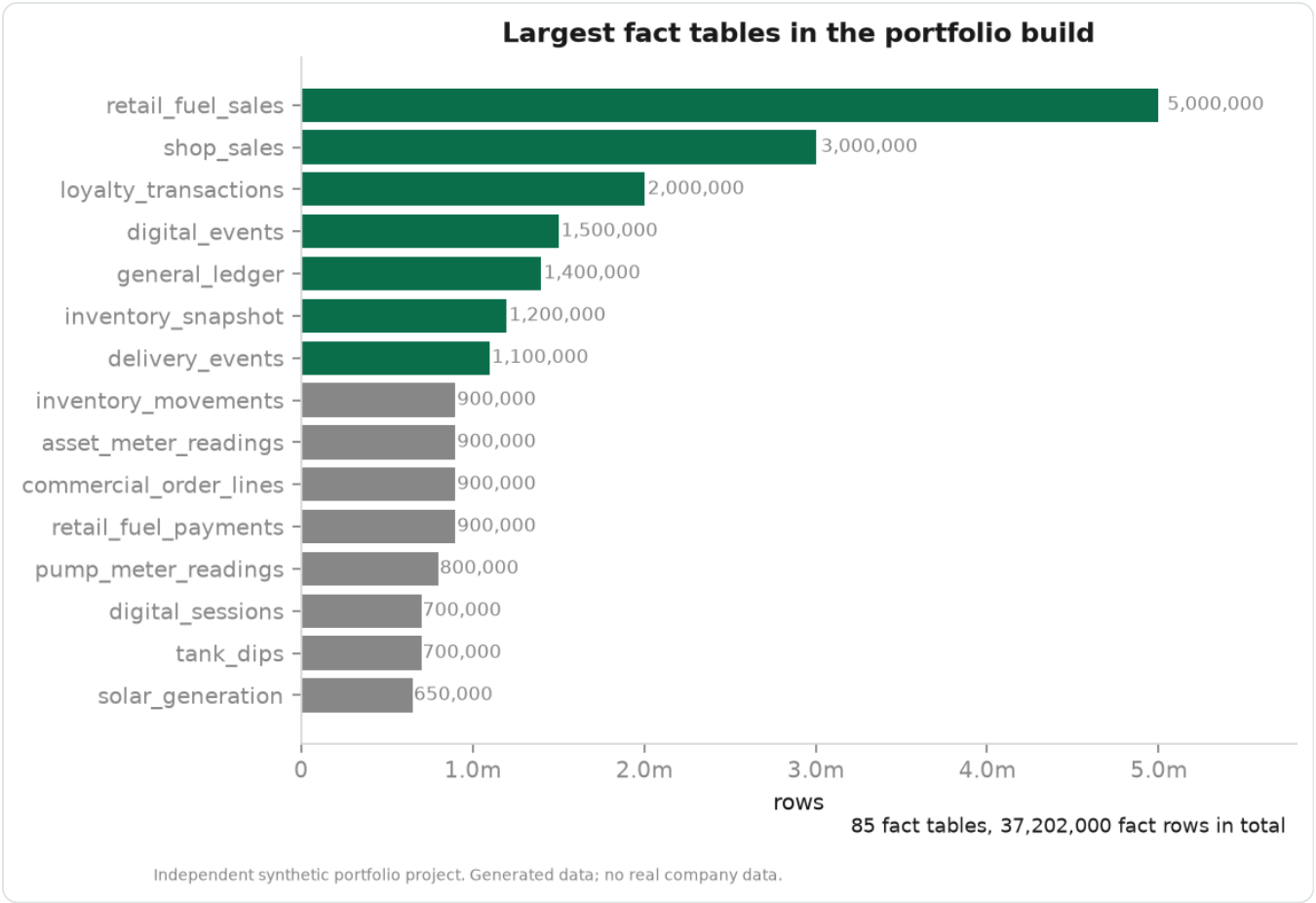
FACT TABLES	FACT ROWS	DEFECTS INJECTED
85	37,202,000	62,924,526
PROVINCES		
9		

The platform covers retail forecourt, commercial B2B, supply and distribution, LPG, lubricants, aviation, marine, loyalty, digital, EV and solar, asset maintenance, HSSEQ and finance — a downstream network of 587 synthetic service stations across 26 African markets, of which 250 are in South Africa spanning all nine provinces. It is built to demonstrate the parts of data engineering that are hard, not the parts that demo well: a deliberately dirty landing zone, a cleansing layer whose output is measured rather than asserted, and models reported against baselines they have to beat.

Build

Table	Rows
fact_retail_fuel_sales	5,000,000
fact_shop_sales	3,000,000

Table	Rows
fact_loyalty_transactions	2,000,000
fact_digital_events	1,500,000
fact_general_ledger	1,400,000
fact_inventory_snapshot	1,200,000
fact_delivery_events	1,100,000
fact_inventory_movements	900,000
fact_asset_meter_readings	900,000
fact_commercial_order_lines	900,000
fact_retail_fuel_payments	900,000
fact_pump_meter_readings	800,000
fact_digital_sessions	700,000
fact_tank_dips	700,000
fact_solar_generation	650,000
fact_customer_invoices	620,000
fact_site_energy_consumption	600,000
fact_customer_receipts	560,000
fact_commercial_orders	550,000
fact_mobile_payments	520,000
fact_site_daily_operations	500,000
fact_loyalty_points_balance	480,000
fact_finance_revenue_cost	420,000
fact_forecourt_shift	420,000
fact_lpg_sales	400,000



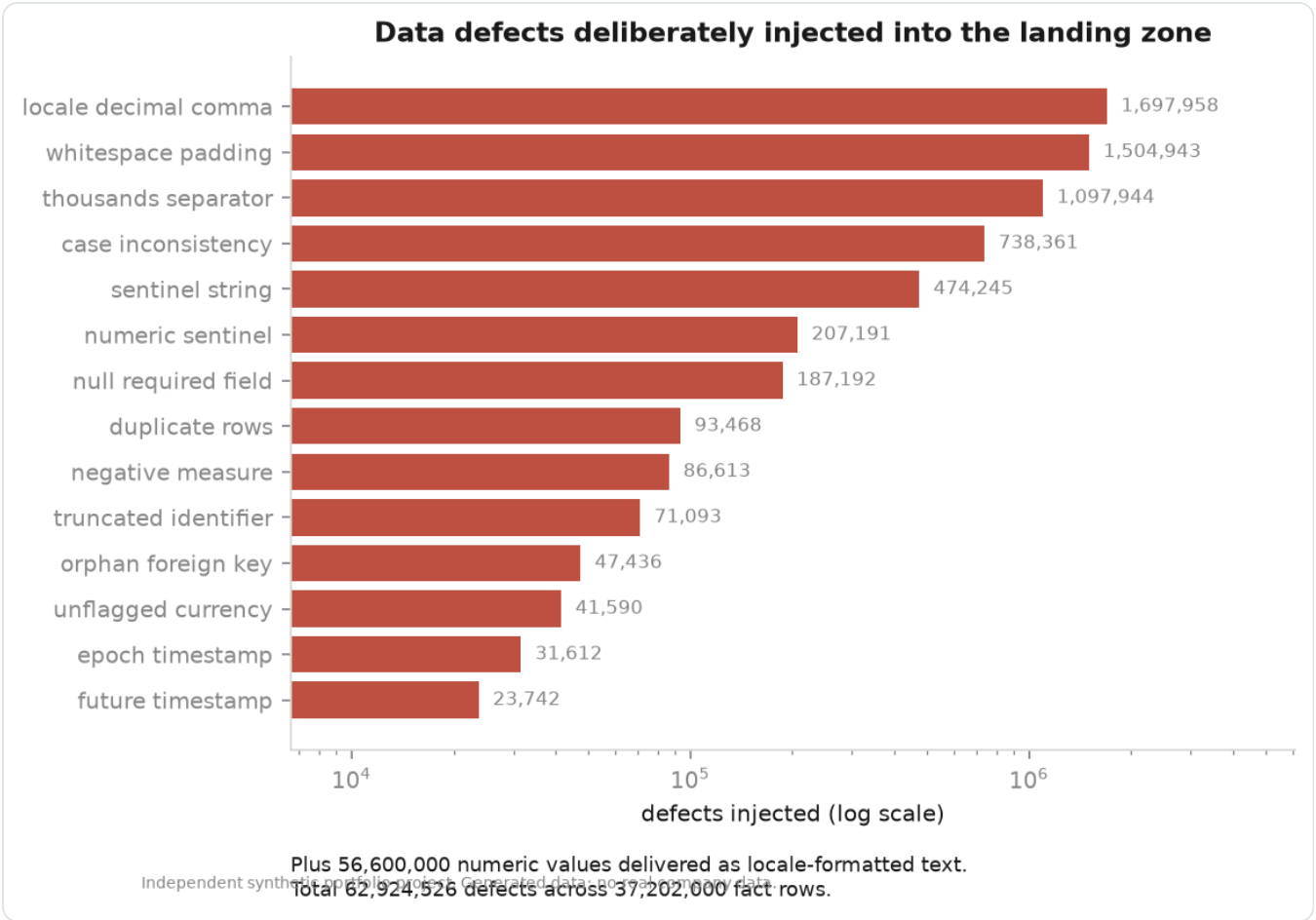
Largest fact tables in the portfolio build.

Data quality

The landing zone is dirty on purpose. A portfolio project built on perfect data demonstrates nothing, because the difficult part of the job is deciding what to do with the row where litres reads "1 234,5", the site code has a trailing space, and the same transaction arrived twice because the point-of-sale replayed its batch. Every defect is counted as it is injected, which turns cleansing from an assertion into a measurement.

Defect type	Defects
numeric delivered as text	56,600,000
locale decimal comma	1,697,958
whitespace padding	1,504,943
thousands separator	1,097,944

Defect type	Defects
case inconsistency	738,361
sentinel string	474,245
numeric sentinel	207,191
null required field	187,192
duplicate rows	93,468
negative measure	86,613
truncated identifier	71,093
orphan foreign key	47,436
unflagged currency	41,590
epoch timestamp	31,612
future timestamp	23,742
derived value mismatch	12,256
unit of measure error	7,233
encoding mojibake	1,649



Defects deliberately injected into the landing zone.

Cleansing outcome

Table	Raw rows	Cleansed	Quarantined	Duplicates removed
fact_retail_fuel_sales	200,000	194,933	4,572	495
fact_shop_sales	120,000	116,702	2,992	306
fact_general_ledger	84,000	83,256	540	204
fact_loyalty_transactions	80,000	78,944	857	199
fact_delivery_events	66,000	65,557	283	160
fact_digital_events	60,000	59,593	236	171
fact_retail_fuel_payments	54,000	53,382	489	129
fact_commercial_order_lines	54,000	53,145	728	127
fact_asset_meter_readings	54,000	53,453	397	150

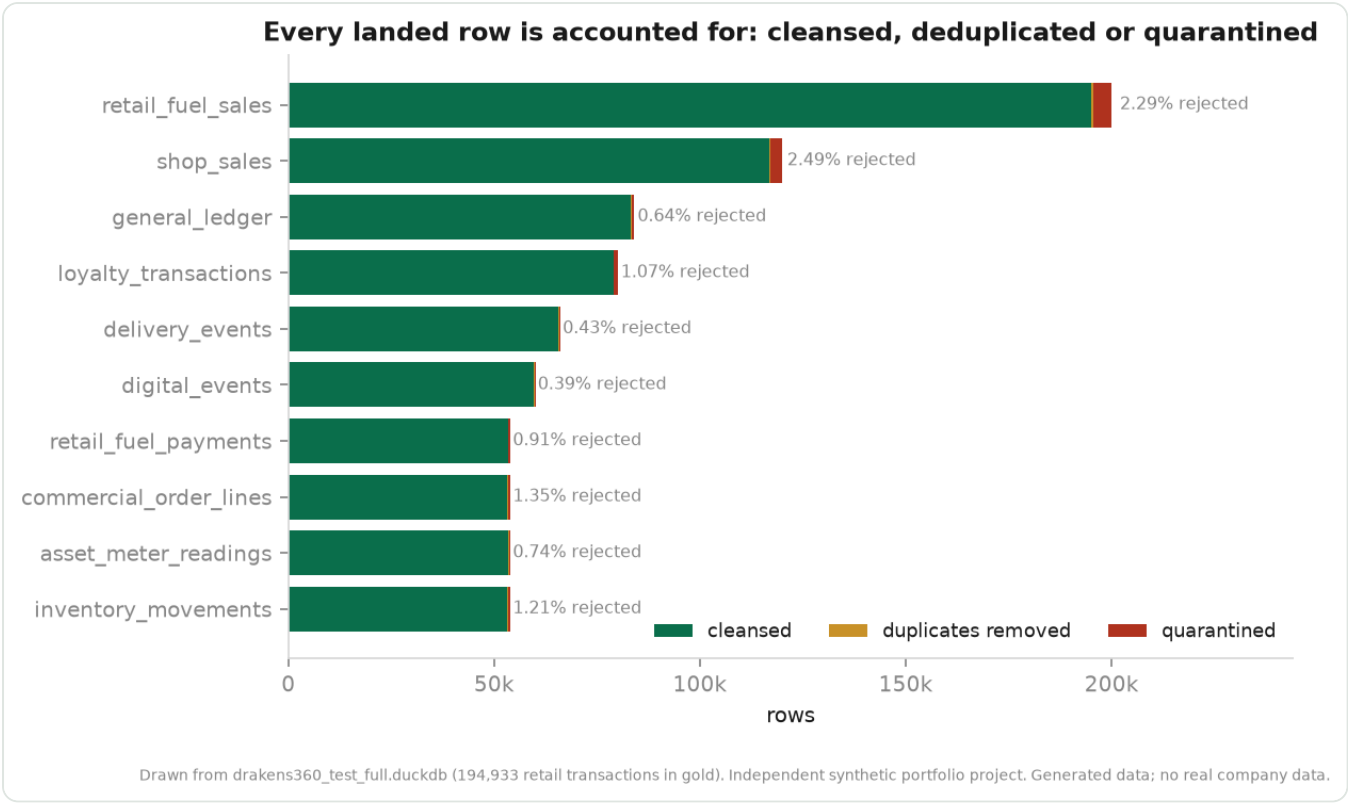
Table	Raw rows	Cleansed	Quarantined	Duplicates removed
fact_inventory_movements	54,000	53,201	656	143
fact_inventory_snapshot	50,000	49,300	583	117
fact_pump_meter_readings	48,000	47,383	502	115
fact_tank_dips	42,000	41,351	543	106
fact_digital_sessions	42,000	41,745	162	93
fact_customer_invoices	37,200	36,733	374	93
fact_site_energy_consumption	36,000	35,623	305	72
fact_customer_receipts	33,600	33,205	301	94
fact_mobile_payments	31,200	30,842	284	74
fact_site_daily_operations	30,000	29,556	365	79
dim_loyalty_member	30,000	30,000	0	0
fact_loyalty_points_balance	28,800	28,491	232	77
fact_solar_generation	26,000	25,704	228	68
fact_finance_revenue_cost	25,200	22,476	2,662	62
fact_forecourt_shift	25,200	24,915	217	68
dim_digital_user	25,000	25,000	0	0
fact_commercial_orders	25,000	24,417	528	55
fact_ev_charger_status	24,000	23,737	199	64
fact_grid_import_export	22,800	22,586	152	62
fact_commercial_deliveries	22,800	22,447	293	60
fact_accounts_receivable	22,800	22,508	232	60
fact_daily_executive_kpi	22,800	22,385	357	58
fact_vehicle_trips	20,400	20,247	100	53
fact_opex	19,200	18,995	164	41

Table	Raw rows	Cleansed	Quarantined	Duplicates removed
fact_customer_credit_exposure	18,000	17,735	234	31
fact_accounts_payable	18,000	17,848	102	50
fact_forecast	18,000	17,739	222	39
fact_lpg_cylinder_movements	18,000	17,814	143	43
fact_lpg_sales	16,000	15,627	331	42
fact_price_changes	15,600	15,353	204	43
fact_budget	15,600	15,479	87	34
fact_replenishment_orders	15,600	15,362	192	46
fact_campaign_responses	15,600	15,470	92	38
fact_training_compliance	15,600	15,442	123	35
fact_deliveries	15,000	14,796	175	29
fact_budget_vs_actual	14,400	14,262	102	36
fact_lubricant_sales	14,000	13,687	281	32
fact_terminal_issues	12,600	12,463	95	42
fact_asset_inspections	12,000	11,879	93	28
fact_spare_parts_usage	11,400	11,292	83	25
fact_procurement_orders	11,400	11,268	110	22
fact_safety_observations	10,800	10,715	55	30
fact_fleet_costs	10,800	10,704	59	37
fact_terminal_throughput	10,800	10,686	93	21
fact_procurement_receipts	10,000	9,858	109	33
fact_energy_costs	9,600	9,470	111	19
fact_lubricant_orders	9,000	8,882	94	24
fact_route_performance	9,000	8,926	56	18

Table	Raw rows	Cleansed	Quarantined	Duplicates removed
fact_permit_to_work	9,000	8,910	73	17
bridge_customer_contract	8,947	6,000	0	2,947
fact_terminal_receipts	8,400	8,314	64	22
fact_aircraft_uplifts	7,800	7,729	49	22
fact_lpg_inventory_snapshot	7,200	7,126	60	14
fact_contract_pricing	7,200	7,123	62	15
fact_cashflow	7,200	7,125	61	14
fact_asset_downtime	7,200	7,117	69	14
fact_inventory_adjustments	7,200	7,110	74	16
fact_lubricant_inventory_snapshot	6,600	6,513	75	12
dim_customer	6,000	5,852	148	0
bridge_customer_hierarchy	6,000	300	0	5,700
fact_ev_charging_sessions	6,000	5,886	102	12
fact_working_capital	5,400	5,337	44	19
fact_vehicle_safety_events	5,400	5,352	34	14
fact_contract_volume_commitments	5,400	5,337	50	13
fact_aviation_sales	5,000	4,897	88	15
dim_asset	5,000	5,000	0	0
bridge_asset_site	5,000	5,000	0	0
bridge_employee_site	4,256	3,000	0	1,256
fact_customer_service_cases	4,200	4,154	38	8
fact_asset_failures	4,200	4,142	46	12
fact_lpg_bulk_deliveries	4,200	4,155	33	12
fact_marine_sales	4,000	3,921	71	8

Table	Raw rows	Cleansed	Quarantined	Duplicates removed
dim_pump	4,000	4,000	0	0
fact_maintenance_work_orders	4,000	3,958	38	4
fact_stock_losses	3,600	3,550	35	15
fact_shop_returns	3,600	3,548	41	11
fact_bunker_deliveries	3,600	3,571	18	11
bridge_site_product	3,536	600	0	2,936
dim_contract	3,500	3,500	0	0
bridge_customer_site	3,172	1,500	0	1,672
dim_employee	3,000	3,000	0	0
fact_near_misses	2,760	2,729	24	7
dim_tank	2,500	2,500	0	0
fact_capex	2,400	2,369	26	5
fact_promotions	2,400	397	223	1,780
fact_aviation_inventory_snapshot	2,400	2,365	27	8
bridge_route_site	1,869	600	0	1,269
fact_marine_inventory_snapshot	1,800	1,776	16	8
fact_environmental_events	1,320	1,296	18	6
dim_equipment	1,250	1,250	0	0
bridge_supplier_product	1,206	400	0	806
dim_driver	1,100	1,100	0	0
bridge_site_promotion	1,018	600	0	418
dim_vehicle	900	900	0	0
fact_hsseq_incidents	800	795	4	1
bridge_site_hierarchy	600	586	14	0

Table	Raw rows	Cleansed	Quarantined	Duplicates removed
dim_route	600	600	0	0
dim_site	600	586	14	0
fact_import_cargoes	540	538	2	0
dim_ev_charger	400	400	0	0
dim_supplier	400	385	15	0
dim_solar_asset	300	300	0	0
dim_city	88	88	0	0
dim_terminal	60	60	0	0
dim_promotion	44	44	0	0
dim_depot	40	40	0	0
dim_campaign	38	38	0	0
dim_product	31	31	0	0
bridge_product_hierarchy	31	31	0	0
dim_price_zone	30	30	0	0
dim_country	29	28	1	0
dim_currency	25	25	0	0
dim_carrier	22	22	0	0
dim_warehouse	20	20	0	0
dim_airport	12	12	0	0
dim_legal_entity	12	12	0	0
dim_province	9	9	0	0
dim_port	8	8	0	0
dim_lpg_cylinder_type	4	4	0	0



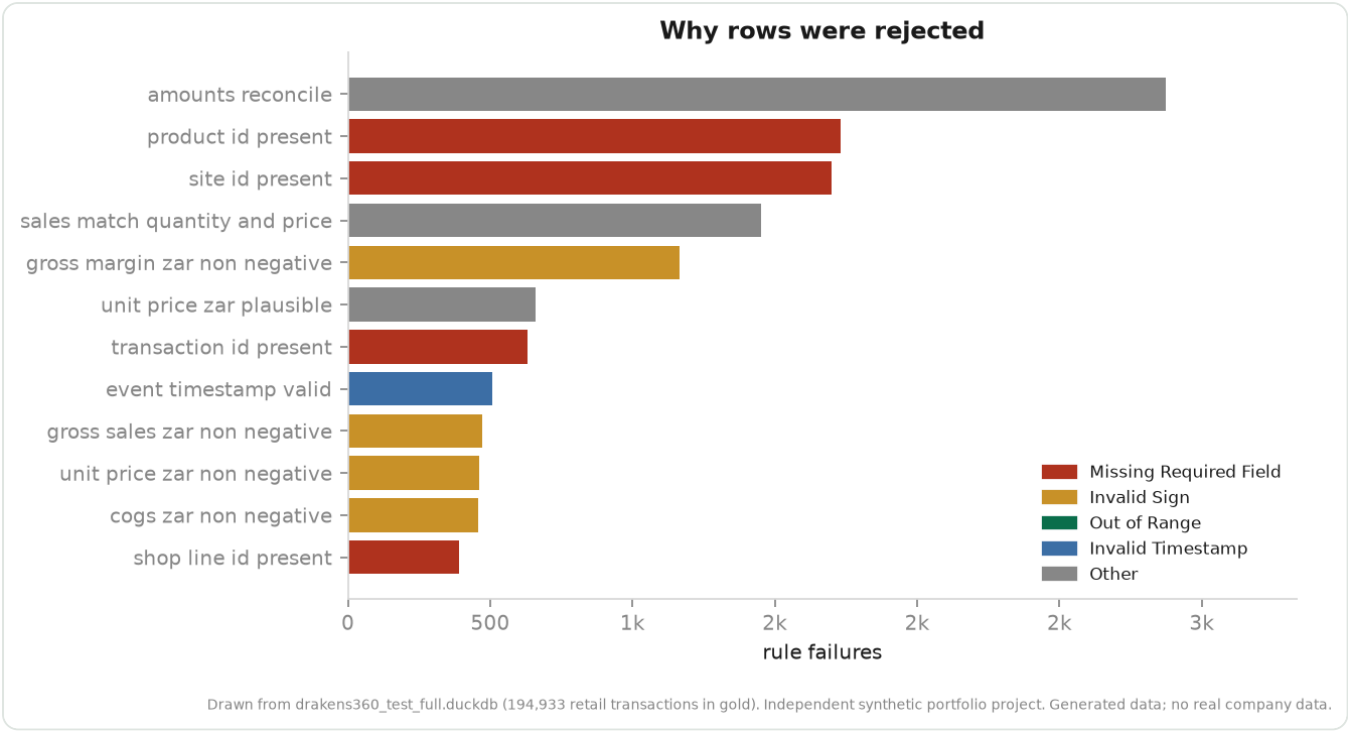
Every landed row is accounted for as cleansed, deduplicated or quarantined.

Why rows were rejected

Rule that rejected the row	Category	Likely owner	Rows
amounts_reconcile	Other	Data Platform	2,873
product_id_present	Missing Required Field	Source System	1,731
site_id_present	Missing Required Field	Source System	1,699
sales_match_quantity_and_price	Other	Data Platform	1,452
gross_margin_zar_non_negative	Invalid Sign	Business Process	1,166
unit_price_zar_plausible	Other	Data Platform	660.00
transaction_id_present	Missing Required Field	Source System	630.00
event_timestamp_valid	Invalid Timestamp	Source System	506.00
gross_sales_zar_non_negative	Invalid Sign	Business Process	473.00
unit_price_zar_non_negative	Invalid Sign	Business Process	460.00
cogs_zar_non_negative	Invalid Sign	Business Process	458.00

Rule that rejected the row	Category	Likely owner	Rows
shop_line_id_present	Missing Required Field	Source System	391.00
journal_line_id_present	Missing Required Field	Source System	311.00
loyalty_transaction_id_present	Missing Required Field	Source System	284.00
litres_non_negative	Invalid Sign	Business Process	253.00
digital_event_id_present	Missing Required Field	Source System	197.00
reading_id_present	Missing Required Field	Source System	176.00
litres_within_range	Out of Range	Data Entry	161.00
snapshot_id_present	Missing Required Field	Source System	160.00
fill_pct_in_range	Other	Data Platform	143.00
tank_dip_id_present	Missing Required Field	Source System	125.00
points_value_zar_non_negative	Invalid Sign	Business Process	117.00
debit_amount_zar_non_negative	Invalid Sign	Business Process	93.00
solar_reading_id_present	Missing Required Field	Source System	91.00
order_id_present	Missing Required Field	Source System	91.00
qualifying_spend_zar_non_negative	Invalid Sign	Business Process	90.00
credit_amount_zar_non_negative	Invalid Sign	Business Process	88.00
volume_litres_non_negative	Invalid Sign	Business Process	88.00
customer_id_present	Missing Required Field	Source System	88.00
capacity_litres_non_negative	Invalid Sign	Business Process	83.00
stock_on_hand_litres_non_negative	Invalid Sign	Business Process	69.00
volume_litres_within_range	Out of Range	Data Entry	59.00
delivery_id_present	Missing Required Field	Source System	56.00
revenue_zar_non_negative	Invalid Sign	Business Process	43.00
energy_kwh_non_negative	Invalid Sign	Business Process	39.00

Rule that rejected the row	Category	Likely owner	Rows
ev_session_id_present	Missing Required Field	Source System	33.00
energy_kwh_within_range	Out of Range	Data Entry	32.00
fill_rate_pct_in_range	Other	Data Platform	28.00
list_price_zar_non_negative	Invalid Sign	Business Process	27.00
delivered_litres_non_negative	Invalid Sign	Business Process	19.00
delivered_litres_within_range	Out of Range	Data Entry	13.00
work_order_id_present	Missing Required Field	Source System	8.00
grid_cost_zar_non_negative	Invalid Sign	Business Process	8.00
labour_cost_zar_non_negative	Invalid Sign	Business Process	7.00
total_cost_zar_non_negative	Invalid Sign	Business Process	7.00
rated_power_kw_non_negative	Invalid Sign	Business Process	7.00
average_power_kw_non_negative	Invalid Sign	Business Process	3.00
parts_cost_zar_non_negative	Invalid Sign	Business Process	3.00
incident_id_present	Missing Required Field	Source System	2.00
estimated_cost_zar_non_negative	Invalid Sign	Business Process	1.00



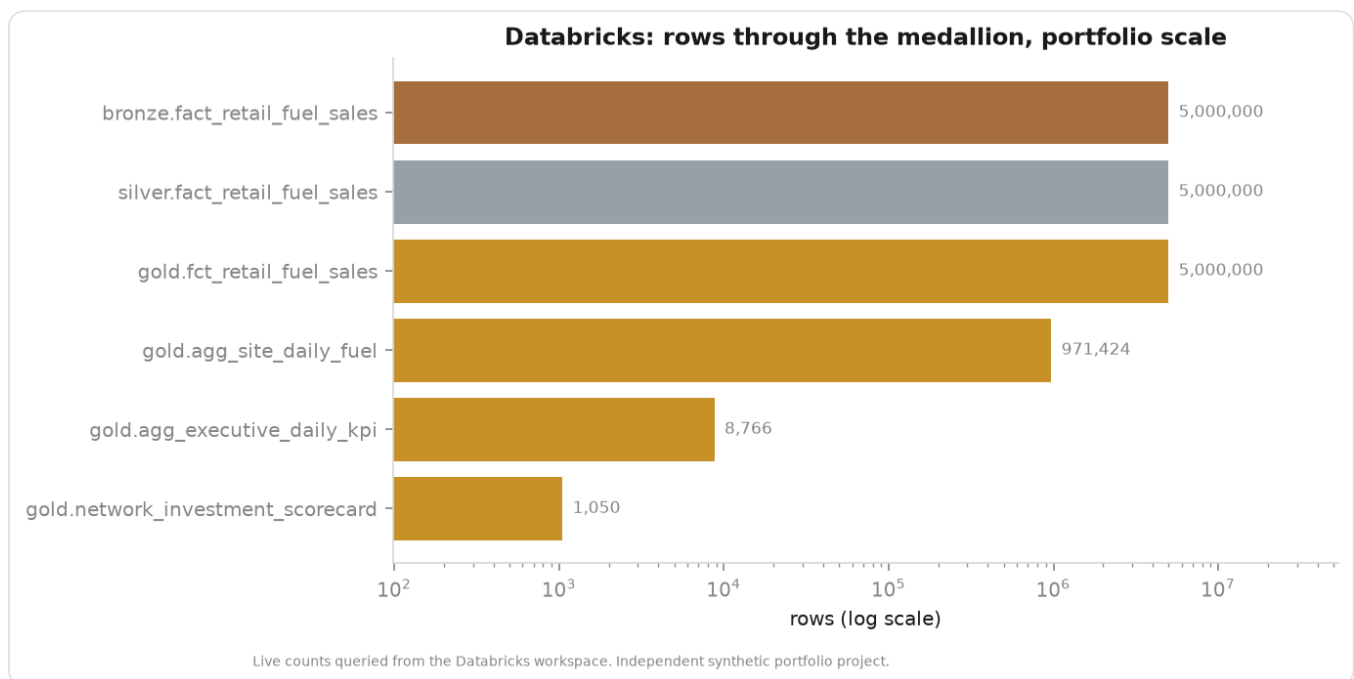
Rejection reasons. The operational response differs by rule.

Warehouse

Control	What it checks	Owner	Variance %	Result
DELIVERIES_HAVE_KNOWN_DESTINATION	Every delivery resolves to a site in the conformed dimension	Supply	0.00	PASS
GL_DEBITS_EQUAL_CREDITS	Posted journal debits equal credits across the ledger	Finance	0.00	PASS
ORDERS_HAVE_KNOWN_CUSTOMER	Every commercial order resolves to a customer in the dimension	Commercial	0.00	PASS
RETAIL_MARGIN_RECONCILES	Retail fuel gross margin equals revenue less cost of sales	Finance	0.00	PASS

Control	What it checks	Owner	Variance %	Result
RETAIL_VOLUME_TIES_TO_DAILY_AGG	Transaction-level litres tie to the site-day aggregate	Supply	0.00	PASS

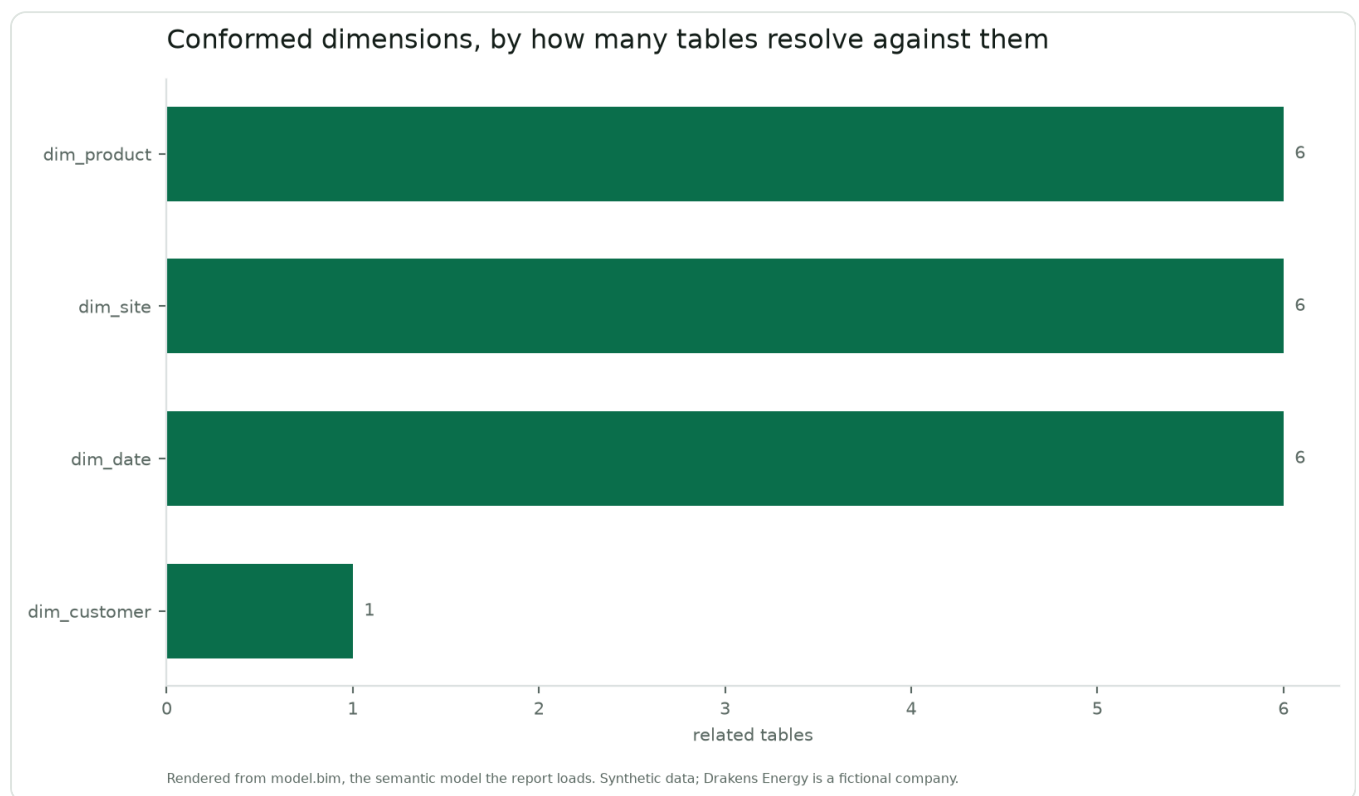
A reconciliation control evaluates a published number as data rather than only as a test, so a control that fails names the figure that is wrong and by how much. Controls that fail are shown as failures; a report that quietly omits them would defeat the purpose of having them.



Live row counts through bronze, silver and gold on Databricks at portfolio scale.

The data model

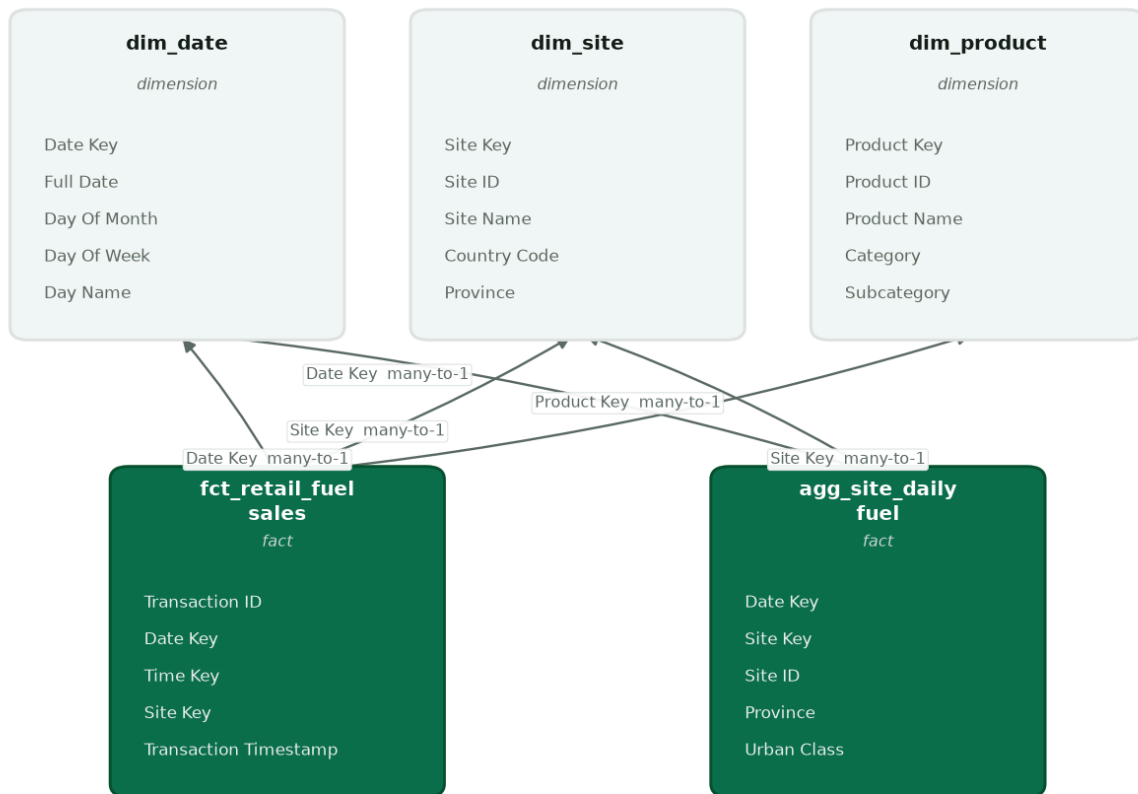
Every diagram below is rendered from `model.bim`, the semantic model the Power BI report loads — not drawn by hand and not transcribed from a design document. Each box, edge and cardinality is therefore a statement about the artifact that was actually built, and a relationship added tomorrow appears in tomorrow's diagram. Where a fact has no modelled relationship the diagram says so, because that absence is a finding rather than a layout accident.



Conformed dimensions ranked by how many tables resolve against them. A dimension that only one fact uses is not conformed; it is a lookup table with ambitions.

Retail fuel

2 fact table(s), 3 related table(s)

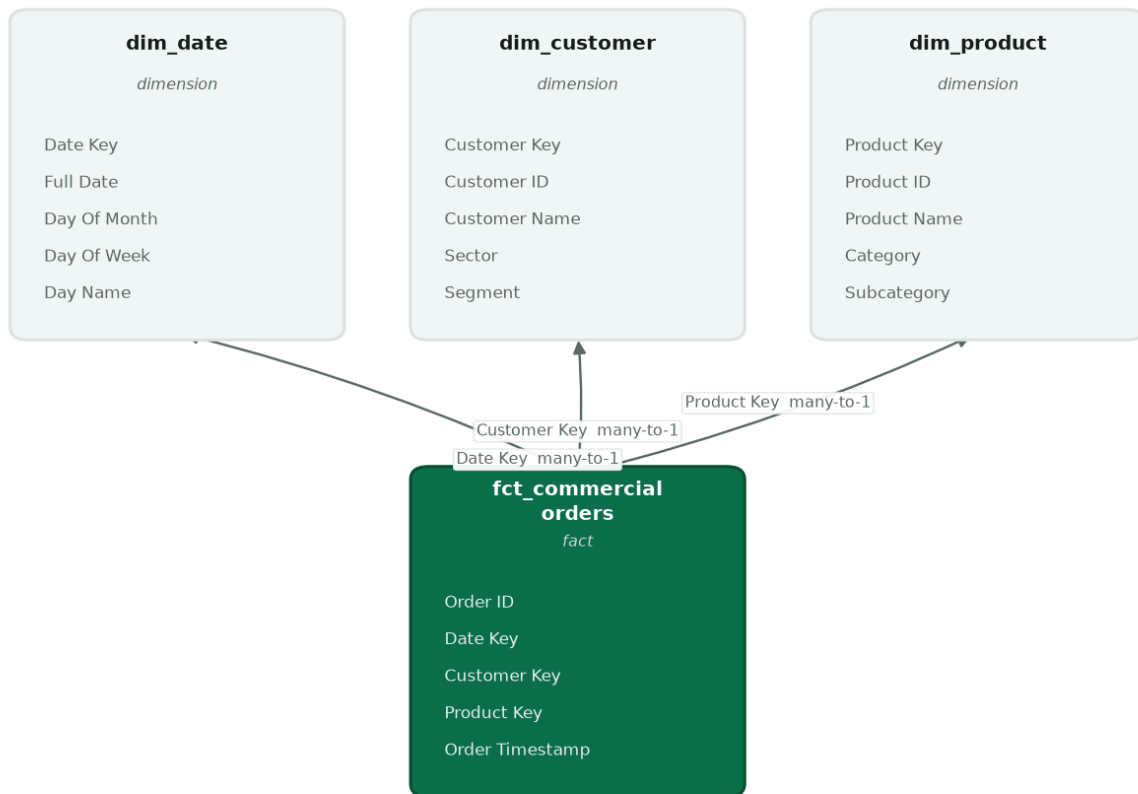


Rendered from model.bim, the semantic model the report loads. Synthetic data; Drakens Energy is a fictional company.

Retail fuel. The transaction fact and the site-day aggregate resolve against the same three dimensions, which is what makes the aggregate reconcilable against the detail rather than merely similar to it.

Commercial B2B

1 fact table(s), 3 related table(s)



Rendered from model.bim, the semantic model the report loads. Synthetic data; Drakens Energy is a fictional company.

Commercial B2B. The relationship to dim_product was missing until recently, which meant lubricants — the largest line in the business at R20.3bn — could not be sliced by product at all.

Supply and distribution

1 fact table(s), 2 related table(s)



Rendered from model.bim, the semantic model the report loads. Synthetic data; Drakens Energy is a fictional company.

Supply and distribution. Deliveries resolve to a destination site, so a delivery to an unrecognised site lands on the unknown member rather than vanishing from the count.

Network investment and geography

2 fact table(s), 1 related table(s)

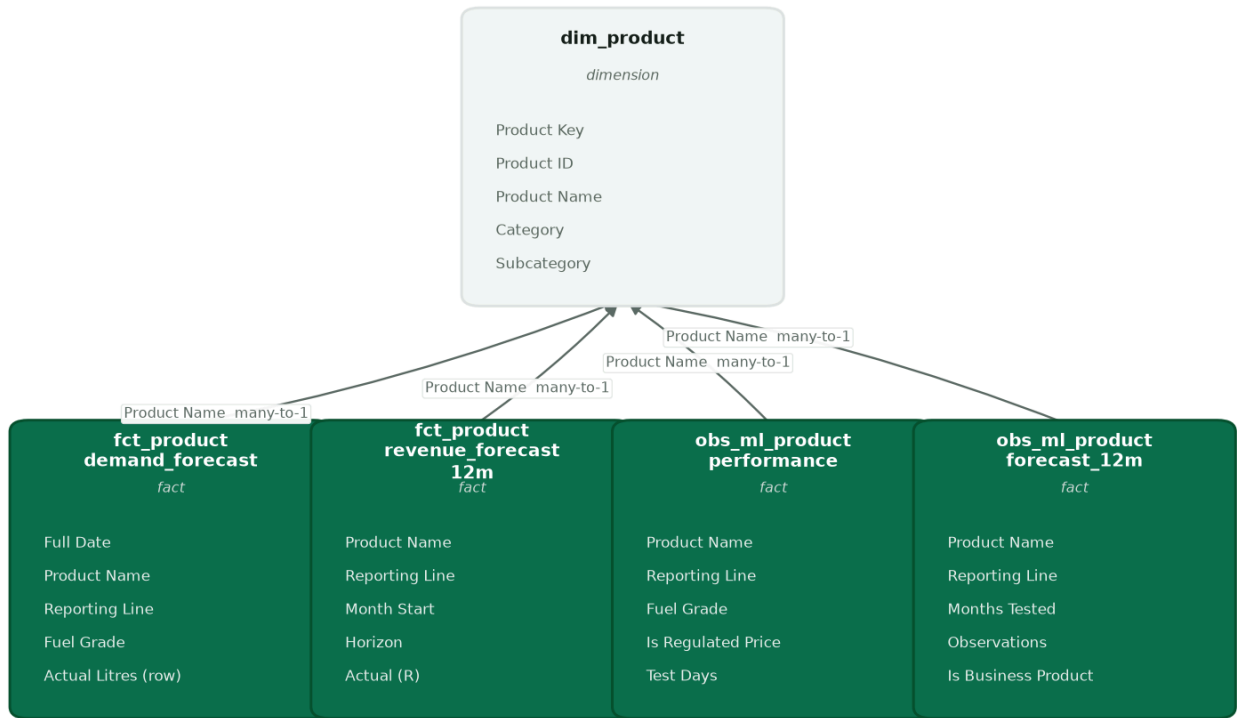


Rendered from model.bim, the semantic model the report loads. Synthetic data; Drakens Energy is a fictional company.

Network investment and the spatial analysis, both keyed on the site dimension.

Forecasting and model results

4 fact table(s), 1 related table(s)



Rendered from model.bim, the semantic model the report loads. Synthetic data; Drakens Energy is a fictional company.

Forecasting and model results. These tables key on product name rather than a surrogate, because they are produced by the ML layer rather than by dbt.

What the diagrams do not cover

Reporting line	Products	Revenue (R)	Status	Note
Lubricant	7	20,256,345,787	Reportable	nan
Road Fuel	4	3,166,357,716	Reportable	nan
Unknown	1	34,595,944	Reportable	nan
Other Fuel	1	4,104,049	Reportable	nan
Aviation	2	0.00	No gold fact	nan
Convenience	8	0.00	No gold fact	nan
Energy	3	0.00	Gold fact exists, no product mapping	no product_key; EV sessions are keyed by charger and site
LPG	4	0.00	No gold fact	nan

Reporting line	Products	Revenue (R)	Status	Note
Marine	2	0.00	No gold fact	nan

Five of the nine reporting lines cannot be charted, and for three different reasons. Four have no gold fact at all. Energy has one — `fct_ev_charging_sessions`, carrying R1.67m of revenue and 232,029 kWh — and no `product_key`, so it cannot join the product dimension. Reading a blank bar as “no data” collapses three different problems into the wrong one.

Implementation walkthrough

How the platform is built, stage by stage, with the decision at each one and the evidence it produced. Command output shown below is captured from actual runs and is labelled as command output; it is not a screenshot of a cloud console.

Databricks implementation walkthrough

Drakens Energy is a fictional business. This portfolio uses generated data, including jittered town-centroid coordinates; it does not represent real service stations or company records.

This walkthrough explains the implementation present in the repository, the decisions it demonstrates, and the evidence needed to validate a run. Source review is not a cloud execution result. Local DuckDB results, Spark notebook results and Power BI captures must retain their own environment, profile, date and commit context.

1. Business questions and data design

The platform connects fuel transactions, sites, products, customers and operational events so a reviewer can follow margin, service reliability, replenishment and investment decisions back to source records. Conformed dimensions allow the same site to be analysed across subjects without separately maintained definitions.

Start with the [implementation guide](IMPLEMENTATION_GUIDE.md), [data dictionary](data_dictionary.md), [subject-area ERDs](erd/README.md), and the [Gold retail ERD](erd/erd_gold_retail_validated.md). State the grain before describing a metric: a transaction line, a site-day aggregate and a site scorecard are different populations.

Evidence to show: an architecture diagram, source inventory, a small source sample, and a model diagram with keys and fact grain. Explain the choice of transactional detail plus aggregates as a trade-off between traceability and report query cost.

2. Select the execution path and scale

The repository contains several paths; they are not interchangeable run records.

Path	Entry point	What the source implements	Evidence required for an execution claim
Local reference build	<code>src/drakens360/build.py</code> , dbt models	Python-generated landing data and a local warehouse used by CI	Selected manifest, warehouse path, command output, commit and test results
Spark-native batch job	<code>scripts/deploy_databricks.py</code> , notebooks 00/02/03/05/06	In-workspace generation followed by conformance, marts, quality and maintenance	Workspace job/run ID, task outcomes and actual output counts
File ingestion	Notebook 01	Auto Loader configuration for landed feeds	Source file, checkpoint/schema location and completed ingestion evidence
Streaming example	Notebook 04	Demo rate source and a separate Kafka source branch	Mode, actual source schema, query progress and restart/late-event evidence

The notebook 00 SCALES dictionary currently configures portfolio generation with 1,050 ZA sites and 4,200 total sites. The previously inspected local report warehouse contains 250 ZA sites and 587 total sites. These are different implementations and profiles. Do not replace one with the other's total or say that all code generates the same estate. Measure the particular artifact used by each figure.

3. Workspace and catalog preparation

`[scripts/deploy_databricks.py](../scripts/deploy_databricks.py)` selects a configured authentication profile, uses an existing SQL warehouse, creates an environment catalog and schemas, uploads notebooks, and creates or resets the named job. `--run` triggers execution; deployment alone does not demonstrate a successful run. The code selects the first returned SQL warehouse, so a reproducibility record must identify which warehouse was actually used.

`[01_catalog_and_schemas.sql](../databricks/sql/01_catalog_and_schemas.sql)` also defines managed volumes, including landing and checkpoints. Notebook upload, schema creation and volume preparation are separate concerns: the deployment helper's catalog function creates schemas, not every volume referenced by optional ingestion notebooks. Inspect required locations before attempting that path.

Record the environment, runtime/compute, dependency versions, catalog, notebook revision and selected scale. Use configuration names in screenshots; exclude secret values. Terraform and Bicep files under `infra/` describe an Azure infrastructure option, and should be labelled as provisioned only when deployment evidence exists.

Skill demonstrated: reproducible environment configuration, separation of environments, explicit dependencies and avoidance of credentials in source.

4. Generate and inspect Bronze data

[00_generate_bronze_at_scale.py](../databricks/notebooks/00_generate_bronze_at_scale.py) uses Spark expressions and scale/seed widgets to create synthetic records in the chosen catalog. The local generator is a separate reference path. Inspect the manifest for the selected local build rather than assuming a notebook run produced that manifest.

Show a small, named input sample with examples of injected defects: numeric text, whitespace, sentinels, duplicate business keys or broken references. Explain which layer will handle each defect. Raw row counts and defect counts describe different things because one record can contain several defects.

Capture: parameters, completed generation output, table preview and the corresponding manifest where applicable. A generated analytical plot is useful here but does not replace execution evidence.

5. Explain the optional Auto Loader path

[01_bronze_autoloader.py](../databricks/notebooks/01_bronze_autoloader.py) has a feed registry for forecourt, shop, commercial, finance and maintenance inputs. It constructs per-feed checkpoint and schema paths and configures `cloudFiles` options including `addNewColumns`, a rescued-data column and a micro-batch file limit.

Document the exact feed, input format, source path, schema location, checkpoint and trigger mode used. Show that a second ingestion of the same input does not unexpectedly increase the target and explain what happens when an upstream schema changes. Configuration alone does not prove restart behavior or uninterrupted schema evolution.

The default generated batch job does not include notebook 01. Keep it visibly separate in the workflow diagram, and do not imply it executed because the five-task batch job succeeded.

6. Silver conformance and quarantine

[02_silver_conformance.py](../databricks/notebooks/02_silver_conformance.py) declares contracts per feed. For retail fuel, examples include required transaction/site/product identifiers, plausible litres and unit prices, and gross-margin reconciliation. It records rule outcomes, writes rejected records to

platform.quarantine, ranks records for deduplication using business keys and timestamps, and writes Silver output.

Use the same business key in the before/after example. Show raw values, the applied rule, accepted output or rejection reason, and the associated run. Reconcile the input population with accepted records, quarantined records and duplicate removal using the implementation's actual counting order; do not add overlapping categories as though they were mutually exclusive.

The notebook includes a dimension SCD2 merge implementation. Demonstrate history with a controlled changed attribute and the resulting current/expired records before claiming the behavior is verified. Distinguish storing dimension history from resolving a fact to the correct historical version.

Skill demonstrated: explicit data contracts, traceable cleaning, deterministic deduplication and explainable rejection handling.

7. Gold marts, keys and grain

[03_gold_star_schema.py](../databricks/notebooks/03_gold_star_schema.py) builds Spark-native dimensions and marts. The dbt models provide a separate path under dbt/models/marts/. Both should be explained with their real definitions rather than described as identical merely because table names resemble each other.

The dbt retail fact is one transaction line per transaction_id. It resolves the current site/product dimension by business identifier and maps unmatched keys to the unknown member -1, retaining flags that make drift countable. The site-day aggregate groups by date_key and site_key. Revenue, cost, litres and margin can be summed over compatible populations; a margin percentage should be recomputed from its numerator and denominator.

Important historical limitation: the inspected dbt retail join filters dimensions to is_current; notebook 03 also selects current site records and assigns site_key from site_id. Neither detail should be described as proof of an as-of historical fact-to-dimension join. An event-time interval join and controlled historical test would be required to demonstrate that claim.

See the [validated Gold retail model](erd/erd_gold_retail_validated.md) for logical relationships and the distinction from source-manifest diagrams.

8. Orchestration and failure boundaries

The actual sequence declared by create_job() is:

```
flowchart LR
    A[generate_bronze / notebook 00] --> B[build_silver / notebook 02]
    B --> C[build_gold / notebook 03]
```

```
C --> D[data_quality / notebook 05]
D --> E[optimize_tables / notebook 06]
```

The helper sets one concurrent run and a task timeout. The listed dependencies explain execution order; do not add a schedule, automatic retry policy, Auto Loader or streaming task to this diagram unless the job definition is changed and verified.

Capture: the actual job graph with run ID and final task states. Include a failed task, its diagnosis and rerun evidence where available. Distinguish a repaired local test from a successful remote workflow.

9. Streaming telemetry and its current boundary

[04_streaming_telemetry.py](../databricks/notebooks/04_streaming_telemetry.py) provides tank, fleet, EV and solar examples. Its demo source yields rate-stream value and timestamp columns. The production branch returns raw_json and kafka_ts, while the inspected downstream tank transformation still refers to value and timestamp. A payload decode/schema adapter is therefore a prerequisite to claiming this production branch works end to end.

Explain event time, configured watermarks, checkpoint paths and the intended destination using the actual code. Show query progress and a late-event/restart test only when executed. A demo generator is not evidence of sustained Event Hubs ingestion. Preserve this boundary in the ebook even when the architecture includes Event Hubs as a planned source.

10. Quality, freshness and governance

[05_data_quality_checks.py](../databricks/notebooks/05_data_quality_checks.py) produces operational results, including freshness records with a run ID and evaluation time. It distinguishes old backfill data from recent-feed SLA checks and can fail a critical check according to its widget setting. The inspected MONITORED list contains SLA constants; do not describe them as dynamically loaded reference data without a code change.

[02_governance.sql](../databricks/sql/02_governance.sql) contains group grants, masks and row-filter logic. The SQL file is implementation evidence; proof of effective permissions requires queries under representative identities. Show a permitted result and a restricted result without displaying credentials or unrelated personal data.

Capture: quality output, reconciliation differences, quarantine reasons and effective-access evidence. A green control must remain tied to the run, table and population it checked.

11. Performance and maintenance

[06_optimize_and_vacuum.py](../databricks/notebooks/06_optimize_and_vacuum.py) describes a per-table maintenance plan and captures table statistics. It exposes run_vacuum and a retention setting, rejects retention below its 168-hour floor, and follows quality checks in the batch graph.

Present observed file count, table size, runtime and query behavior only with the relevant before/after capture and workload context. Comments reporting an earlier measurement are historical claims until their supporting artifact is identified. A change in file count alone is not a benchmark of query speed or a monetary saving.

Document the effect of retention and maintenance on recoverability. Do not execute maintenance or provision compute merely to fill a screenshot slot.

12. ML, decision support and report consumption

The experiments in ml/experiments/ and the MLflow export in [export_ml_results.py](../scripts/export_ml_results.py) provide model evaluation evidence. Explain target, grain, train/test split, leakage controls, metric direction and baseline for each model. AUC is higher-is-better; MAE is lower-is-better. The export's raw score-minus-baseline delta has opposite improvement directions for these metrics and should not be ranked on one mixed-unit chart.

The unsupervised anomaly experiment has a different evaluation status from the scored supervised models. Preserve weak results and missing scores. The network investment engine is a constrained optimization component; do not present its rules or allocation as a learned prediction model.

Show how exported evaluation results reach the Power BI model. Evaluation summary rows are not per-site prediction records. For actual prediction use, identify the scored entity/date grain, model/run identity and consuming visual. Power BI page captures must prove the rendered output, and paired captures should show changed metrics when a South Africa filter changes.

13. Reproducibility and portfolio evidence

The implementation guide owns commands and environment setup. [reconcile_figures.py](../scripts/reconcile_figures.py) checks document figures against source artifacts; a fresh successful result belongs beside the final generated PDF/HTML versions. Keep the workflow's Git commit and CI result separate from a Databricks run ID.

Use this coverage register when preparing final document figures:

Evidence	Suitable capture	What it proves
Environment and scale	Configuration plus run header	Which implementation and population were used
Source and Bronze	Actual sample and generation/ingestion result	Input content and completed landing step
Silver	Same-key before/after and quarantine result	Transformation or rejection of identifiable records
Gold and ERD	Schema, key tests and grain checks	Model structure and relationship validity
Job execution	Completed job task graph	Which tasks ran and their outcomes
Streaming	Query progress and controlled late/restart examples	The specific streaming behavior exercised
Quality and governance	Control results and representative access queries	Validation and effective restrictions for that run
ML	Experiment metrics, split details and baseline	Evaluation rather than unsupported accuracy claims
Power BI and Excel	Actual pages, filters and formatted results	Correct and readable consumption
Final build	Reconciliation and completed CI result	Traceability of the delivered version

Authentic UI captures remain outstanding where desktop/workspace access is unavailable. Clearly label captured command output as execution output, analytical charts as charts, and missing UI captures as missing. Number figures in document order and keep captions with images. This walkthrough is source-verified documentation; it does not certify those outstanding runtime checks.

COMMAND OUTPUT — SPATIAL AXIS GUARD

ST_Distance_Sphere axis check: Johannesburg to Cape Town

expected 1,261,576 m

measured 1,261,576 m PASS

point-in-polygon check (against Natural Earth admin-1):

Not South Africa	336
Matches	232
Outside the mapped province polygons	15
Province mismatch	2
No province recorded	1

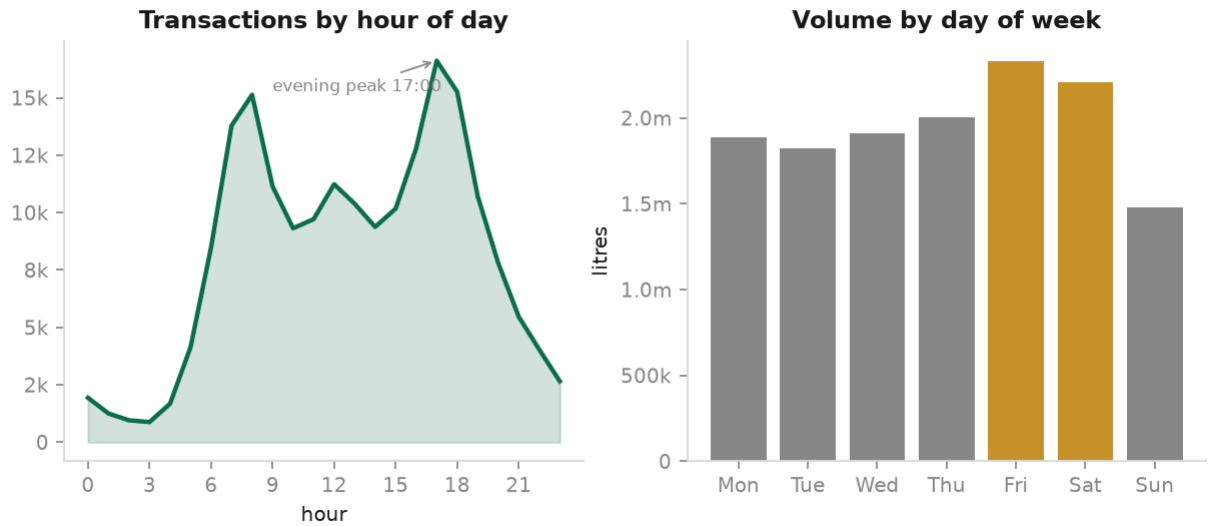
Capture 1 — not available in this environment. The Power BI Forecast page before and after a reporting-line filter, showing the score and forecast panels changing together. Native window capture returns a blank image for Power BI Desktop in this session, so no screenshot is presented in its place.

Capture 2 — not available in this environment. The Databricks job run history for the medallion pipeline. Native window capture returns a blank image for Power BI Desktop in this session, so no screenshot is presented in its place.

Demand and trading rhythm

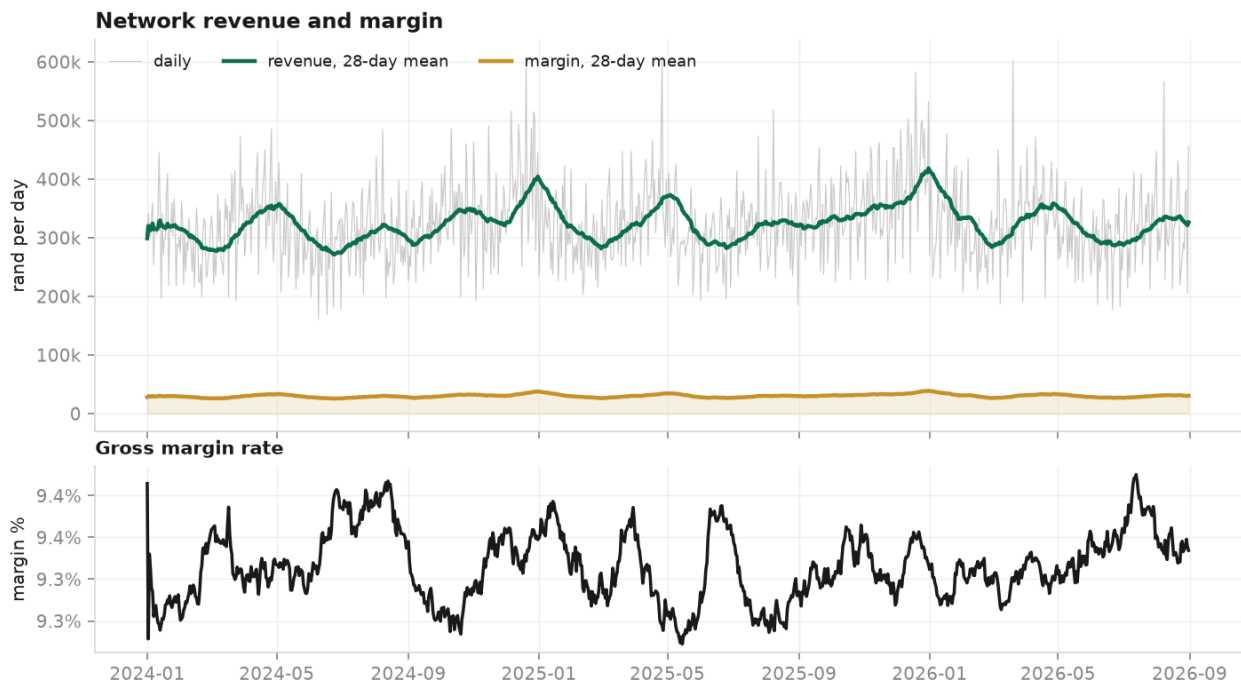
A forecasting model can only learn a pattern the data actually contains. These three figures are the evidence that the generator produces one: an hourly commute shape, a weekly rhythm, and a southern-hemisphere seasonal cycle that survives all the way through to the warehouse.

Generated demand follows real trading rhythms



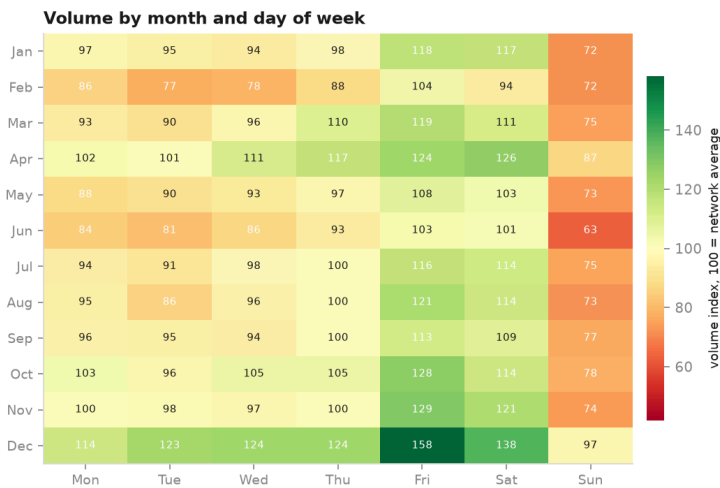
Drawn from drakens360_test_full.duckdb (194,933 retail transactions in gold). Independent synthetic portfolio project. Generated data; no real company data.

Commute peaks at 08:00 and 17:00, and a Friday-heavy week.



Drawn from drakens360_test_full.duckdb (194,933 retail transactions in gold). Independent synthetic portfolio project. Generated data; no real company data.

Revenue and margin across the modelled window, with the margin rate on its own axis. A rate and a total share an axis only in charts that say nothing about either.



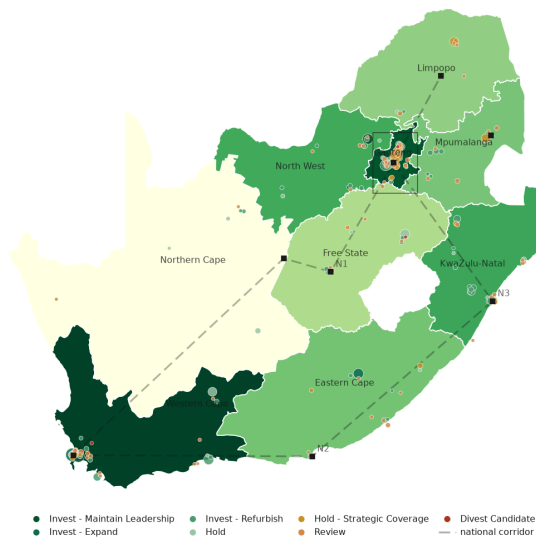
Mean litres per trading day, indexed to 100. Per day rather than per month because the modelled window ends in August: summed, September to December would read a third low and look like seasonality.

Drawn from drakens360_test_full.duckdb (194,933 retail transactions in gold). Independent synthetic portfolio project. Generated data; no real company data.

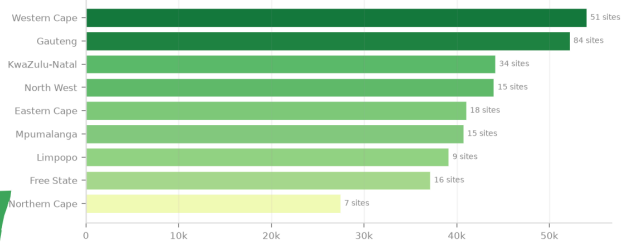
Month against day of week, indexed to the network average. December and January are the high-summer holiday peak; June and July the winter trough.

Network and investment

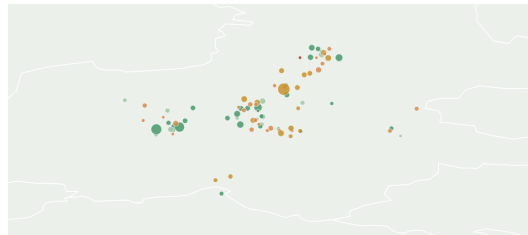
Recommendation, margin and corridor exposure



Margin per site, by province



Gauteng detail (84 sites)



250 synthetic sites across 9 provinces – the South African estate; 250 of 987 sites; the remaining 337 are in other African markets and have no province. Bubble area is total margin; colour is the investment recommendation. Province boundaries are Natural Earth 1:10m admin-1 (public domain); everything drawn on them is generated. Coordinates are South African town centroids plus random jitter and do not represent any real service station. Drawn from drakens360_test_full.duckdb (194,933 retail transactions in gold). Independent synthetic portfolio project. Generated data; no real company data.

Provinces shaded by margin per site; bubble area is total margin and colour is the investment recommendation. The N1, N2 and N3 are drawn because corridor sites are protected from divestment regardless of score. Province boundaries are Natural Earth 1:10m admin-1, public domain; everything drawn on them is generated.

Measure	Value
Budget	R1.50bn

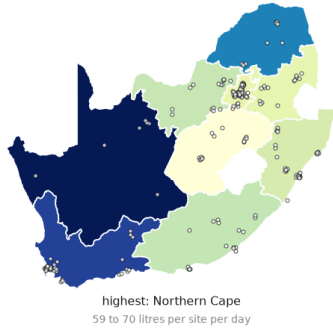
Measure	Value
Funded projects	248
Capital committed	R1.330bn
Expected annual uplift	R332.9m
Portfolio ROI	25.0% per year
Blended payback	4.0 years
Provinces covered	9 of 9

Plan by province

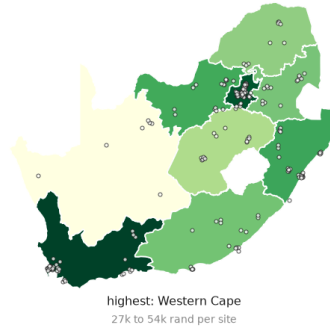
Province	Projects	Capex (R m)	Uplift (R m)
Gauteng	84	439.20	109.11
Western Cape	51	279.30	79.15
KwaZulu-Natal	34	188.00	44.93
Eastern Cape	16	87.30	21.48
Free State	16	85.60	19.52
North West	15	85.00	21.03
Mpumalanga	15	79.30	16.77
Northern Cape	7	42.20	11.74
Limpopo	9	38.20	8.19

The same network, read six ways

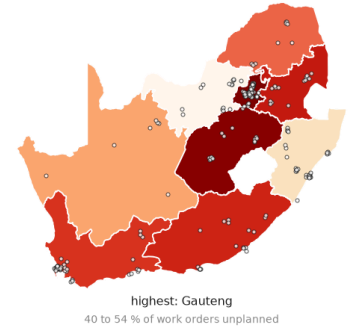
Throughput



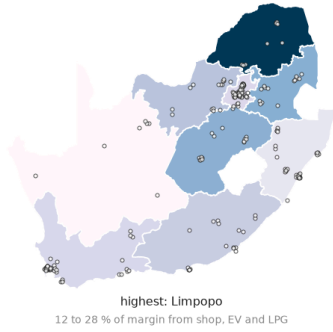
Margin per site



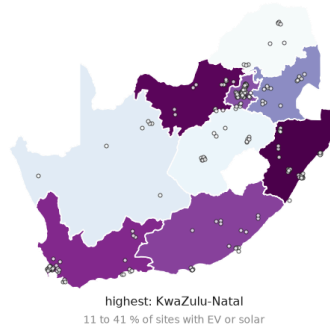
Asset condition



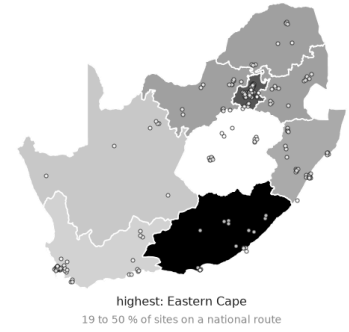
Non-fuel margin



New energy



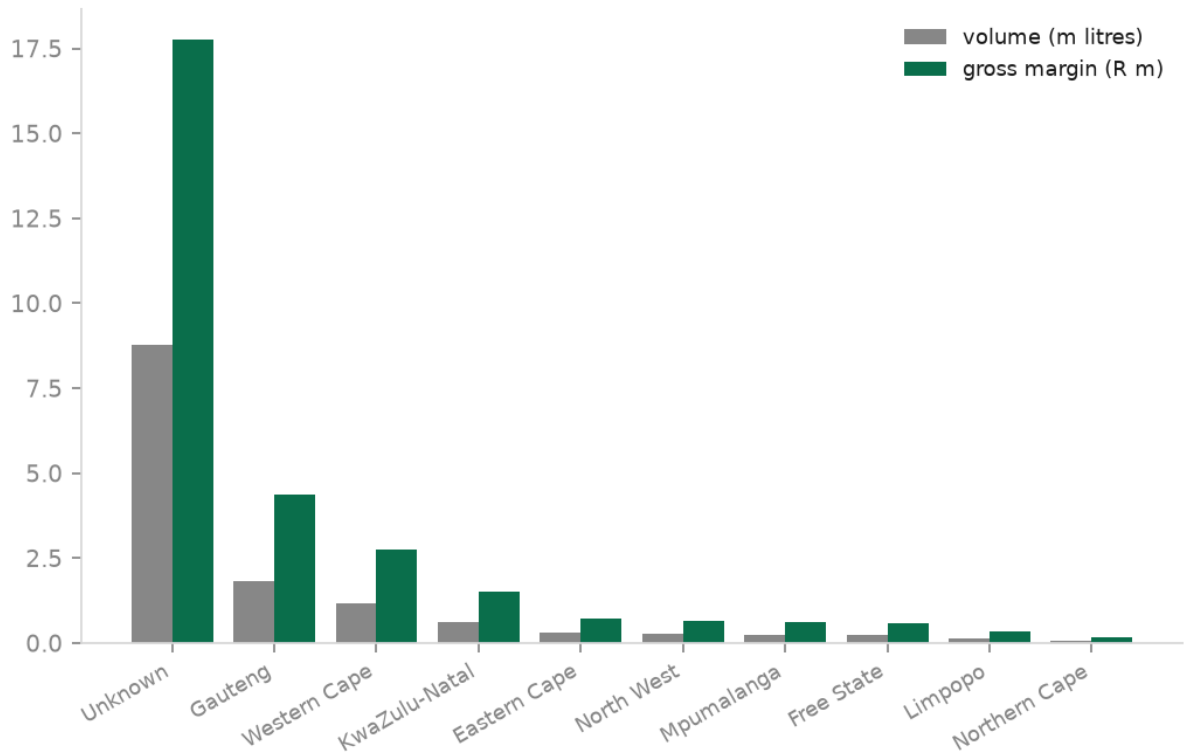
Corridor exposure



Provinces shaded by the panel's measure; dots are the synthetic sites, identical in every panel so the six are comparable. Province boundaries are Natural Earth 1:10m admin-1 (public domain). Drawn from drakens360_test_full.duckdb (194,933 retail transactions in gold). Independent synthetic portfolio project. Generated data; no real company data.

The same network read six ways, on a fixed basemap and extent so the panels are directly comparable. A single map answers one question; a network planner asks six.

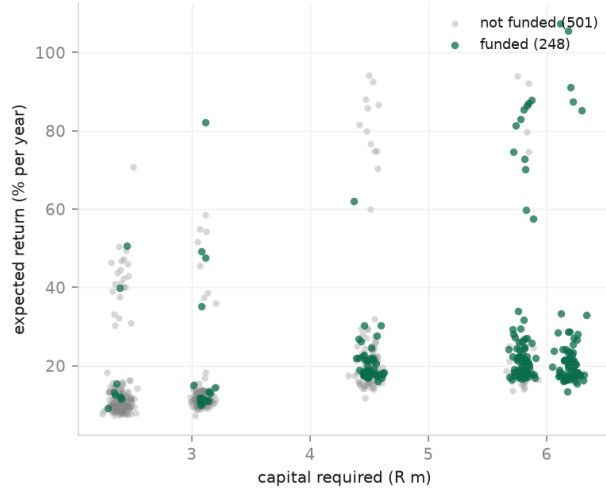
Volume and margin by province



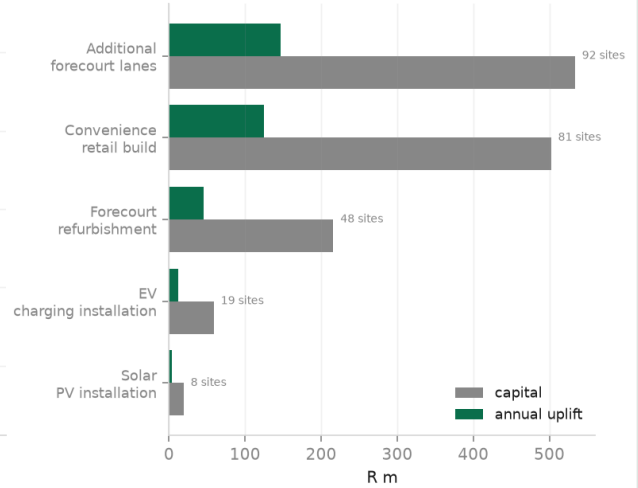
Drawn from drakens360_test_full.duckdb (194,933 retail transactions in gold). Independent synthetic portfolio project. Generated data; no real company data.

Volume and margin by province.

Candidate projects: return against capital

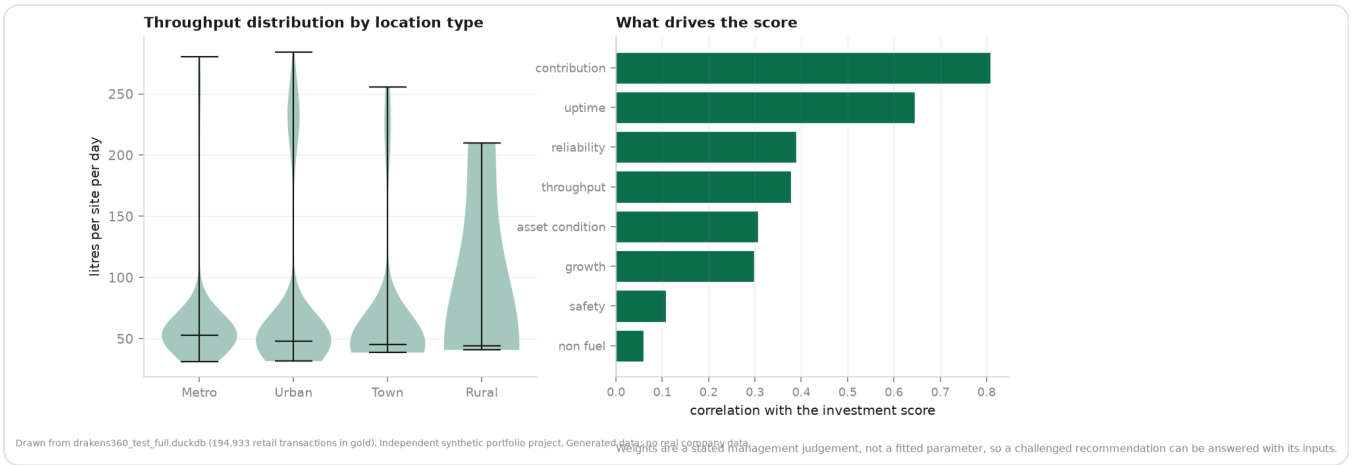


Funded plan by intervention



Drawn from drakens360_test_full.duckdb (194,933 retail transactions in gold). Independent synthetic portfolio project. Generated data; no real company data.

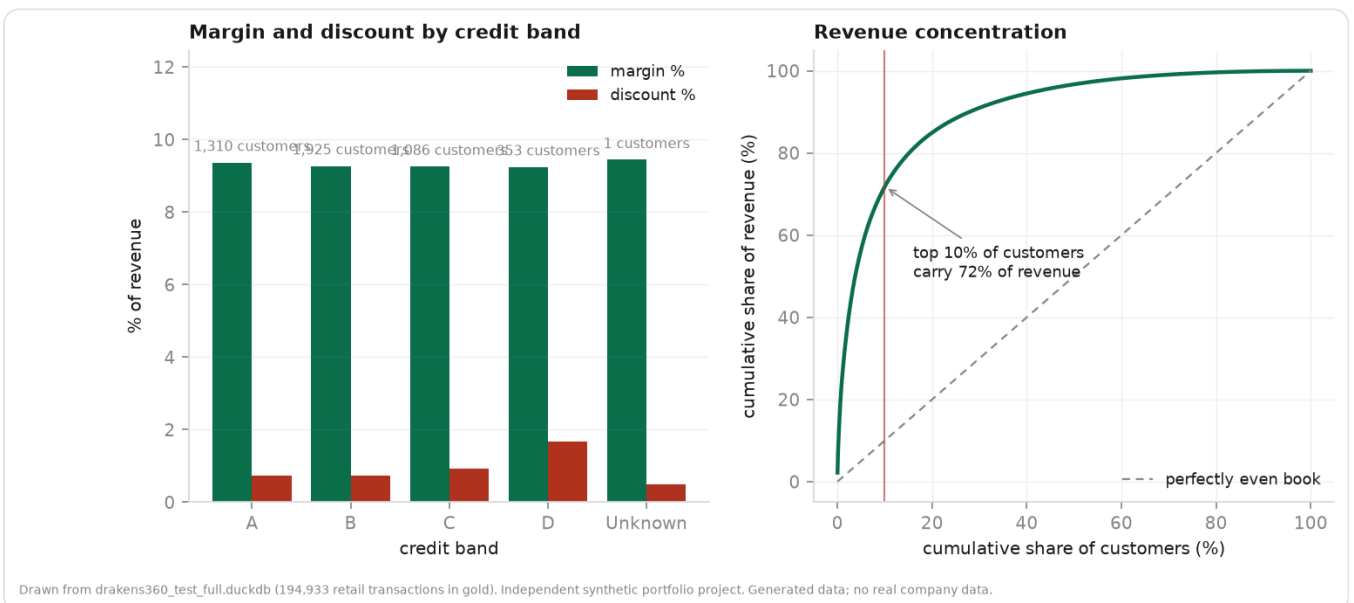
Every candidate project by return against capital, with the funded set highlighted. The funded set is not simply the top of the ROI ranking — provincial minimums and corridor protection pull specific projects in.



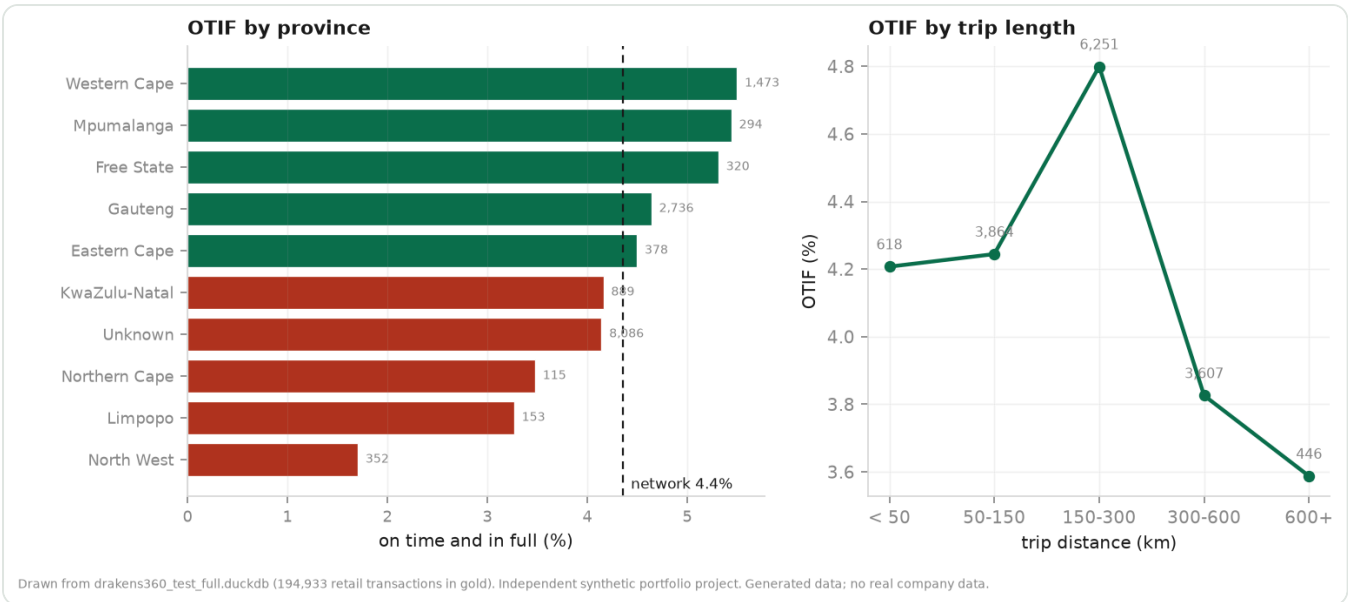
Left: throughput is skewed enough that a network average is a poor summary. Right: what actually drives the investment score, as the correlation between each component and the total.

Commercial and operations

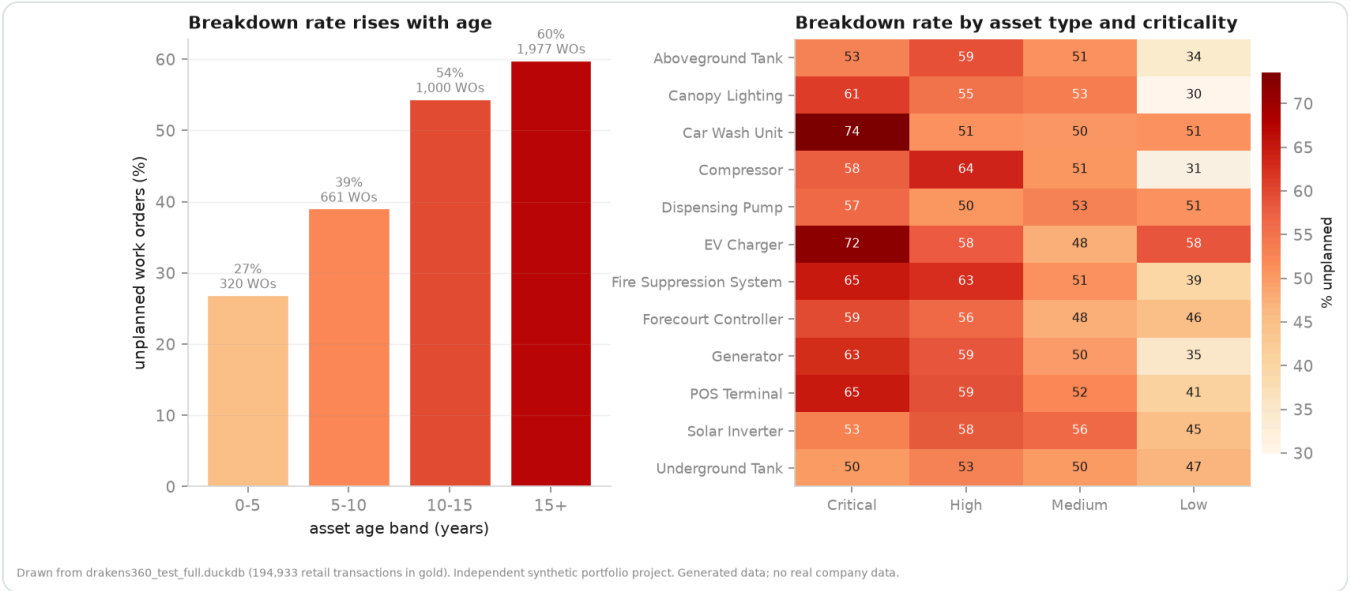
Four questions the business asks that the platform is built to answer, each computed once in gold rather than reassembled per report.



Left: whether the credit band means anything — worse-rated customers discount harder. Right: how concentrated the customer book is, which is a different risk from an even one.



OTIF broken out by province and by trip length. As a single network number it is the least useful form of it: broken out, it says whether lateness is a routing problem, a distance problem or one region’s problem.

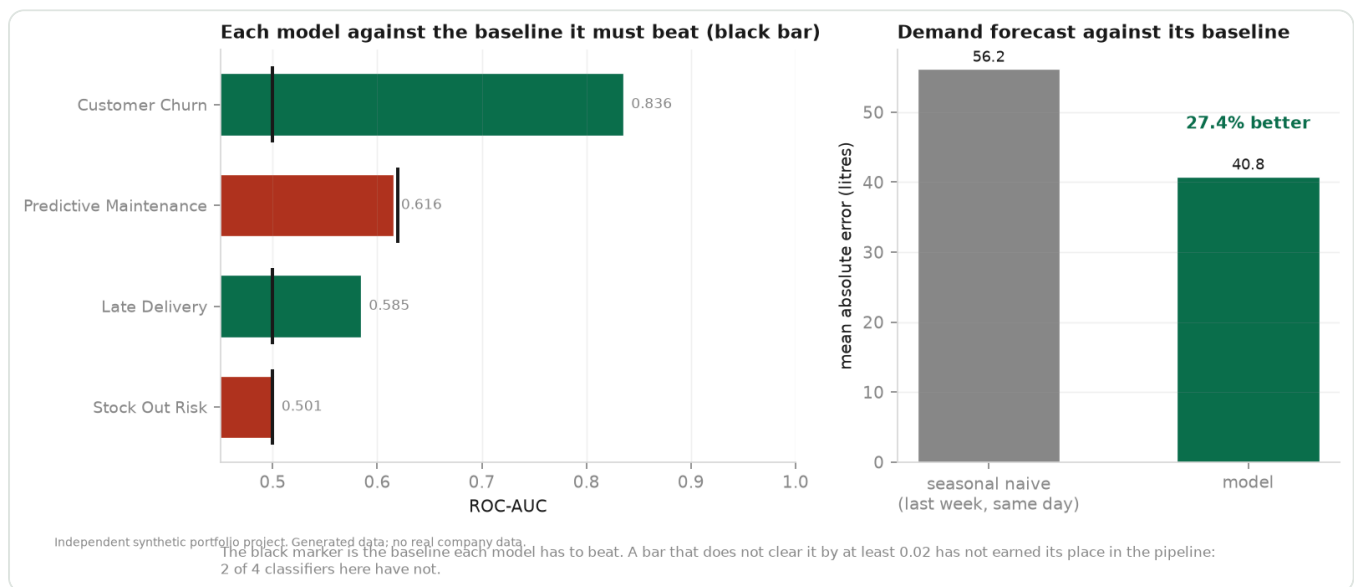


Breakdown rate against asset age and criticality. This is the signal the predictive-maintenance model is allowed to learn from, and the reason that experiment reports an age-only baseline.

Machine learning

Six experiments, each reported against a baseline it has to beat. Not all of them win, and all six are listed — a portfolio that shows only the models that worked is not showing the job.

Model	Headline result	Verdict
Customer Churn	ROC-AUC 0.836, 82% precision in the top decile	Usable — ranks well above chance
Demand Forecast	MAE 40.8 L against a seasonal-naive 56.2 L	Beats its baseline by 27.4%
Fraud Anomaly	1.0% of shifts flagged; cash share 30.9% against 23.9% for normal shifts	Flagged shifts differ from normal ones on both axes
Late Delivery	ROC-AUC 0.585, 1.72x lift in the top decile	Usable — the reviewed decile is enriched
Predictive Maintenance	ROC-AUC 0.616, 1.19x lift in the top decile, against a baseline of 0.620	Does NOT beat its baseline
Product Demand Forecast	not run	not run
Product Revenue Forecast 12M	not run	not run
Stock Out Risk	ROC-AUC 0.501	No usable signal



Each classifier against the baseline it has to beat, and the demand forecast against its seasonal-naive baseline. A bar that does not clear its black marker has not earned its place in the pipeline.

What the platform found in itself

Each of these was caught by the project's own tests, observability or baselines, which is the point of having them.

A macro that silently corrupted every ratio

`safe_divide('a - b', 'c')` compiled to `a - (b / c)` because the macro did not parenthesise its arguments. Not an error — a plausible-looking wrong number. Caught by a range test asserting that `dim_product.base_margin_pct` falls between -5 and 100.

A general ledger that did not balance

Journal lines were posted independently, so debits exceeded credits by 46%. Caught by `obs_reconciliation_controls`, which evaluates double-entry as data rather than only as a test. Fixed by emitting journals as balanced pairs.

A partitioning choice that looked obvious and was wrong

The retail fact was partitioned on `date_key`, because every query filters on a date range. The maintenance job measured the result: 974 files averaging 0.2 MB, which OPTIMIZE can never merge because it does not cross partition boundaries. Under liquid clustering on `(date_key, site_id)` the same 5 million rows occupy 2 files averaging 94 MB — a 487-fold reduction in file count for identical data.

A leak that scored 0.998

The stock-out model was scored against the current inventory snapshot. Its target is `days_of_cover` below a threshold; `days_of_cover` is stock over usage; and both components were features. Excluding `days_of_cover`, which the code did with a comment explaining why, excluded nothing — the model divided one component by the other and reproduced the rule. Re-pointed at the next snapshot it scores 0.510.

A model with no baseline, losing to one variable

Predictive maintenance reported a 1.35x lift and passed review. It had no baseline. Ranking the queue by asset age alone scores ROC-AUC 0.620 against the model's 0.598, and the reported lift moves between 1.03 and 1.35 across capacity settings with no trend. Every experiment now computes and logs the baseline it has to beat.

Null foreign keys where an unknown member belonged

The landing zone contains referential drift by design — a site code in a fact that the dimension never received. Gold left-joined those facts and kept the null key, which drops the row from every inner join and from any total grouped by province: 5,070 retail transactions with real litres and real margin were invisible to regional reporting. They now resolve to a -1 unknown member, so nothing disappears from a total and the drift is countable.

A rule inferred from a column name, four times over

The pattern recurred often enough to be worth naming. A validity rule written against what a column is *called* rather than what it *means* destroys real data, and does it silently while every model builds and every test passes. `signed_amount_zar >= 0` quarantined every credit leg of the general ledger. A 2023-2027 timestamp window nulled 89% of site opening dates. An 8-45 rand fuel price band rejected 36% of the shop feed, where the same column name carries the price of a pie. Requiring a site on every row quarantined 58% of digital events, which are app sessions and were never at a station. Bands are now keyed by table and column, attribute dates get a window wide enough for a business history, and optional foreign keys are measured rather than assumed.

A tab that split a dimension in two

The maintenance heatmap showed thirteen asset types where the dimension has twelve. `clean_text` called `trim()`, which in both DuckDB and Spark removes spaces and nothing else, so a tab-padded value reached gold as a separate category from the same value without one. Both real, both counted separately, neither obviously wrong in a total.

Observability that measured 8% of what it claimed

`obs_cleansing_summary` named fourteen tables against a landing zone of 167, and the fourteen were whichever tables someone had thought to add — the least useful sample available. It is now generated by the same pass that generates the cleansing models: 128 monitored tables, 2,049,277 raw rows reconciling exactly to cleansed plus quarantined plus deduplicated, and zero tables failing reconciliation.

A validity rule that deleted half the general ledger

The contract applied a blanket rule to every amount column: anything named `*_zar` must be non-negative. `signed_amount_zar` is negative on the credit side by design, so the rule quarantined 41,886 of the ledger's 84,000 lines — every credit leg of every journal. What survived was a ledger of debits with almost no credits, which reported a 99.98% imbalance that looked like a generator bug and was a contract bug. A validity rule inferred from a column name is a guess, and a wrong guess deletes data silently instead of failing loudly. Signed measures are now matched by naming convention and excluded, so a new `net_` or `_variance` column is covered the day it is added.

Double-entry integrity is a property of the journal, not the line

Cleansing assesses rows one at a time, so when one leg of a balanced pair failed its contract the other leg survived alone. Each orphaned leg sat in the ledger as an unmatched debit or credit, and the ledger could not balance however correct the generator was — the imbalance was created by the cleansing step itself. The mart now publishes only journals whose lines still net to zero, and the orphans stay in quarantine beside the leg that was rejected. Debits and credits now agree exactly, to the cent.

A sentinel that passed every non-negativity check

The generator uses 0 as one of its numeric sentinels, and 0 satisfies every 'amount ≥ 0 ' rule. Stripping zero the way -999 is stripped is not an option, because zero litres and zero margin are legitimate readings. The contract now checks amounts against their own row instead: a transaction of 82 litres at R22.34 reporting R0.00 of sales is internally inconsistent whatever caused it.

Limitations

Stated plainly, because a report that claims completeness is misleading.

- The Databricks results are at portfolio scale (5,000,000 retail rows through bronze, silver and gold). The DuckDB warehouse figures in this report are at test scale, and each figure is stamped with the warehouse it came from. The two should not be read as one run.
- Predictive maintenance does not beat an age-only baseline, and stock-out risk has no signal at the current snapshot cadence. Both limits are in the generated data rather than in the estimators, and both fixes are changes to the generator: condition monitoring for the first, denser inventory readings for the second.
- The investment plan's rand figures rest on one stated benchmark — what a mid-sized site earns in fuel gross margin in a year — because the transaction fact is a sample and capital costs are not. Rank order and every ratio between sites are unaffected; only the level is rebased.
- Row filters are correct but their query cost under concurrent executive load has not been measured.
- There is no data subject access or erasure workflow. The masking and pseudonym design supports one; the process itself is not built.

Drakens Energy is a fictional company invented for this project. It does not exist. Every site, customer, employee, supplier, asset, coordinate, price, volume, margin and incident in this report is randomly generated. Site coordinates are South African town centroids plus random jitter and do not represent any real service station.